Software Engineering Intern

Take-home assignment: collections agent and bank reconciliation

Time allowed	What you submit	Tools
2 days (48 hours) from the time our email was sent	GitHub repository, explanation PDF and a short video	Any language and framework, a NoSQL database, AI tools allowed

1 Overview

Sonic AI builds revenue automation software for B2B companies, from signed contract through to invoicing, collections and revenue recognition.

This assignment covers the two steps after an invoice is sent: getting the customer to pay, and confirming that the money arrived. You will build a small, independent version of both. We will not use your work in our product.

The organization

AcmeCorp, a fictional company. Your application works for its Accounts Team.

The customers

Nine fictional companies in four currencies that buy from AcmeCorp, reply by email and pay by bank.

Your application chases overdue invoices by email, answers routine customer questions using AI, and suggests which invoice each bank payment belongs to. A person decides anything that needs judgement.

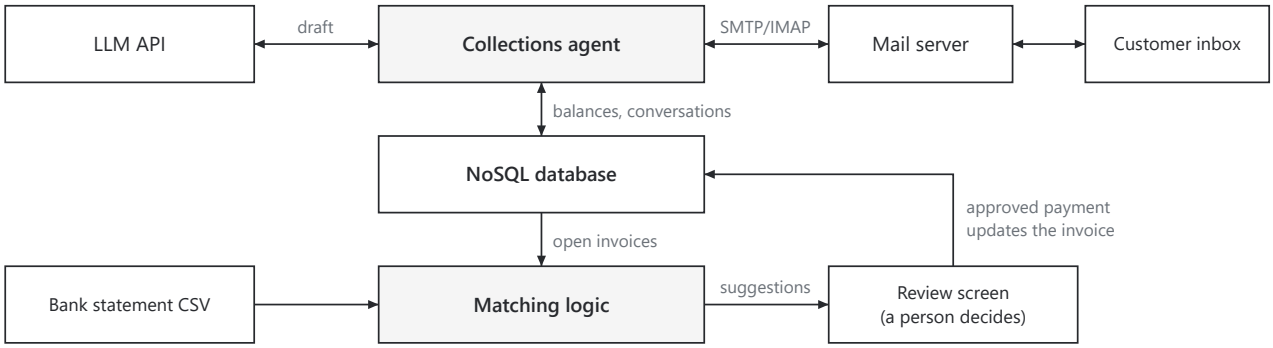


Figure 1. The whole application. Once a payment is approved, the agent stops chasing that invoice.

2 Terms you will need

Draft, Sent, Paid	The stored status of an invoice. A draft has not been sent, so the customer does not owe it yet.
Overdue	Not a stored status. An invoice is overdue when it is sent, has money unpaid, and is past its due date.
Bank reconciliation	Checking each bank statement receipt against the invoices.
Narration	A statement line's description. NEFT, RTGS, IMPS and UPI are Indian transfer methods; SWIFT and SEPA are international.
TDS	Tax Deducted at Source. A customer in India may pay less than the invoice total and pay the difference to the government. Here it is 10% of the subtotal, so a ₹3,54,000 invoice with a ₹3,00,000 subtotal may be settled by paying ₹3,24,000.

3 Data provided

The zip file attached to our email has a `data` folder. Include it in your repository and use the data as given. All of it is fictional. Start with `data-explanation.md`, which explains every file and field.

`organization.json`	AcmeCorp's contacts, bank details and billing policies
`customers.json`	Nine customers, each with a contact person and email address
`invoices.json`	Twenty-seven invoices in INR, USD, EUR and GBP
`bank_statement_sample.csv`	A sample statement. We will test with a different one in the same format.

Customer email addresses use `example.com` and cannot receive mail. Replace them with your own test inboxes while you build.

4 Application structure

We expect a complete application that could be built on later, not a quick demo.

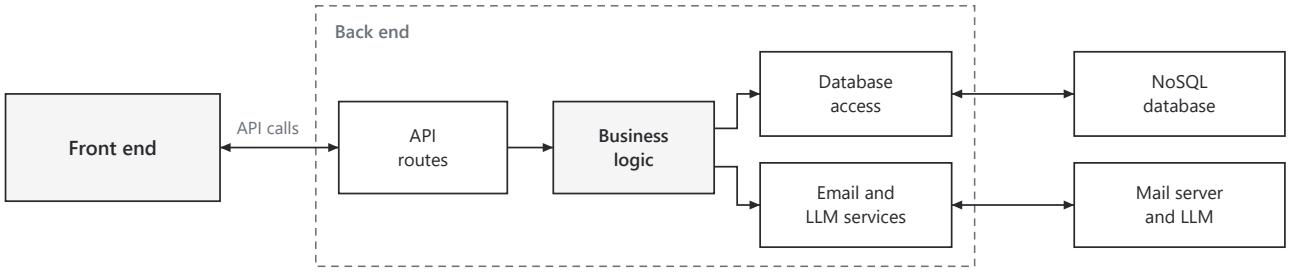


Figure 2. How the application is split.

Front end

- A separate application in a `frontend` folder, such as React or Next.js.
- Talks to the back end only through its API, whichever API style you choose.
- Never touches the database, mail server, LLM or any secret.
- Has pages for the organization, customers, invoices, collections and bank reconciliation.

Back end

- A separate application in a `backend` folder, such as FastAPI or Express.
- Holds all business rules.
- Has a good structure, with routes, business logic, database access and outside services kept apart.
- Has an API that you design and document in your README.

Database

- Store all application data in a NoSQL database. We prefer MongoDB, but any NoSQL database is acceptable.
- Enter the initial data through your front end, or load it from the data files with a script.

Security

- Never commit passwords or API keys. Use environment variables, with a `.env.example` of placeholders.
- Protect against common risks, such as unsafe input and one customer seeing another's data.
- Check for any other security concerns in your application, and handle them too.

5 Part A: Base application

- Show the organization details, customers and invoices, including which invoices are overdue and how much is unpaid on each.
- Allow a customer's email address to be edited. We will enter our own inbox and reply as that customer.
- Provide a **Reset demo data** button that restores the original data and clears conversations and payments.

6 Part B: Collections agent

The agent sends reminders for overdue invoices, reads the replies and answers them. Create two email accounts for testing: one for the organization and one acting as a customer.

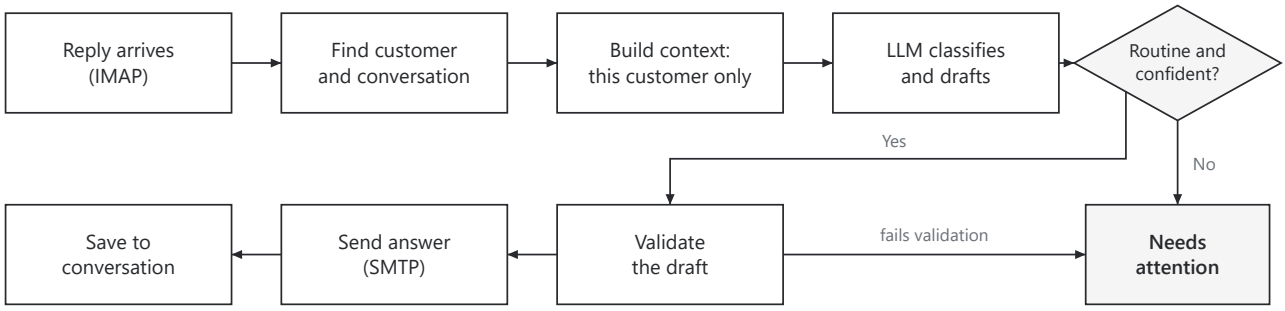


Figure 3. What happens when a customer's reply arrives.

What it must do

- Send reminders using SMTP, with the correct invoice number, amount and due date.
- Read replies using IMAP, automatically and through a **Check inbox now** button.
- Attach each reply to the right customer and conversation.
- Answer using an LLM, with the organization's details and that customer's invoices as context. You do not need RAG or a vector database, because this amount of data fits comfortably in a prompt.
- Make sure every amount, date and invoice number in an email matches the database.
- Show each conversation, and a **Needs attention** list where a person can edit the AI's draft and send it.

How it should respond

If the customer...	The agent should...
Asks about their balance, invoices or how to pay	Answer automatically, with only sent, unpaid invoices
Says they have paid, or will pay on a date	Acknowledge and note it, but never mark it as paid
Asks for more time, disputes, or mentions legal action	Agree to nothing and move it to Needs attention
Asks about another company, or is an unknown sender	Share nothing and move it to Needs attention

Example

Arjun Nair, Kestrel Foods

Hi, is this the only amount pending from us? Also please send your bank details.

Automatic answer from the agent

Dear Arjun, two invoices are outstanding: INV-1004 for ₹59,000.00, due 19 September 2026, and INV-1005 for ₹59,000.00, due 20 October 2026. The total is ₹1,18,000.00. You can pay by NEFT, RTGS or IMPS to AcmeCorp, account 000111222333, Example Bank, IFSC EXMP0000142.

7 Part C: Bank reconciliation

A person uploads a bank statement. The application suggests which invoice each receipt pays, and the person decides.

What it must do

- Upload a CSV in the format of the sample file.
- Match on the payer, amount, currency and dates. A payment's date should make sense against the invoice date.
- For each receipt, suggest the most likely invoice with a confidence score and a one-line reason, or say that there is no match.
- Let a person approve, reject or manually choose a different invoice. Nothing is applied until a person approves it.
- When a payment is less than the invoice, let the person choose **Close invoice (TDS deducted)** or **Leave balance open**.
- Reduce the invoice balance when a payment is approved. At zero, the invoice becomes paid and **the agent stops chasing it**.

Bank reconciliation bank_statement_sample.csv

Date	Narration	Amount	Suggested	Score	Reason	
30 Sep	NEFT CR-BRIGHTLINE LOGISTICS PVT LTD-INV-1002	2,36,000.00	INV-1002	98	Invoice number found, exact amount	Approve Reject
24 Oct	NEFT CR-MAPLE RETAIL SERVICES	12,750.00	None		Payer not recognized	Choose invoice

Figure 4. A rough idea of the review screen. Use your own design.

How to approach the matching

Do not send the statement directly to an LLM and ask it for the answer. Build the matching locally with algorithms first. Fuzzy and semantic matching are the main things we are looking at in this part.

Techniques you could explore include Levenshtein, Jaro-Winkler, token set ratio, Jaccard similarity, n-grams with TF-IDF, and sentence embeddings from a local model. Work out for yourself which ones are best for this problem, and why.

If it is still needed after that, you may use an LLM to improve the output.

8 Bonus items

These are optional, and leaving them out will not count against you.

- **Link a payment email to its bank transaction.** If a customer writes, "We paid INV-1004 yesterday, reference N123456789", show that email beside the matching statement row and use it as extra evidence.
- **One payment covering several invoices.** Suggest the combination and split the payment when it is approved.
- **Out-of-office replies and bounced emails.** Recognize them, so the agent does not answer them like customer emails.

9 Documentation to read

Please read and understand these pages from Stripe and Sequence. We will ask you questions about them.

Stripe: revenue recognition	Overview, How it works, Subscriptions and invoicing, Examples, Reports
Sequence: revenue recognition	Overview, Recognition methods, Deferred revenue, Use cases
Sequence: contracts and schedules	Concepts, Billing concepts, Billing schedules, Phases, Quotes, Renewals
Sequence: pricing models	Product catalog, Standard, Usage-based, Seat-based, Minimum fees
Sequence: invoicing and usage	Invoicing customers, Credit notes, Usage metering

10 What to submit

Within 2 days of receiving our email, reply to it with your repository link and video link.

GitHub repository	A public repository under your own account, with your complete code. Its README must explain how to run everything end to end, including database setup and settings. We will follow it as written, using our own email accounts and LLM key.
Explanation PDF	2 to 3 pages, named `explanation.pdf`, in the main folder of your repository. Explain the algorithms, architecture and tech stack, and how everything was done.
Video	2.5 to 3 minutes explaining the application, shared as an unlisted link.

11 How we will evaluate your work

Our team will run your application and exchange emails with your agent as a customer. We will upload a bank statement you have not seen, and check that everything lines up across the application. We will read your code and explanation, and watch your video. We may call shortlisted candidates to discuss their work and the documentation.

Area	What we look for	Weight
Collections agent	Correct figures and conversations, sound handling of emails	25%
Bank reconciliation	Results on our statement (wrong matches cost more), choice of algorithms	25%
End-to-end behaviour	Runs from your README; an approved payment updates invoice and agent	15%
Architecture and security	Front end and back end split, structure, database design, security	15%
README, explanation, video	Clear instructions, sensible decisions, honest account of what you finished	10%
Documentation concepts	Your understanding of the concepts in Section 9	10%

12 Ground rules

- Submit your own work. Submissions copied from another person or repository will be rejected.
- Find an LLM provider with a free tier or free credits. We do not provide keys or reimburse costs.
- Create new email accounts for this assignment, and never email anyone who has not agreed to take part.
- If something is unclear, reply to our email. Otherwise make a reasonable decision and note it in your README.
- If your work is incomplete, send it anyway and say what is missing.